Experiment 8:
Implement a word count program in Hadoop and Spark.

Step-by-Step Spark Installation on Windows

Prerequisites

1. Java JDK (Java 8)
2. Spark (pre-built with Hadoop)
3. Hadoop binaries (for Windows)
4. WinUtils.exe
5. Set Environment Variables
6. Run Spark (Spark Shell)

1. Install Java JDK

- Download Java JDK (version 8):
  https://www.oracle.com/java/technologies/javase-downloads.html
- Install it and note the path (e.g., C:\Program Files\Java\jdk1.8.0_281).
- Set JAVA_HOME environment variable:
  - Go to **System Properties > Environment Variables**.
  - Add a **new system variable**:
    - Name: JAVA_HOME
    - Value: C:\Program Files\Java\jdk1.8.0_281
  - Add %JAVA_HOME%\bin to the Path.

2. Download and Extract Spark

- Go to: https://spark.apache.org/downloads.html
- Choose:
  - Spark version: Latest stable (e.g., 3.4.1)
  - Package type: *Pre-built for Apache Hadoop 3.3 and later*
- Download and extract the .tgz file to a directory like:
  C:\spark

3. Download WinUtils.exe (Hadoop Support for Windows)

Spark on Windows needs **winutils.exe** to work with Hadoop.

- Download winutils.exe for Hadoop 3.3+ from:
  https://github.com/steveloughran/winutils
  or unofficial mirrors like https://github.com/cdarlint/winutils
- Place winutils.exe into:
  C:\hadoop\bin

4. Set Environment Variables for Spark

Go to **System Properties > Environment Variables**, and add:

- **HADOOP_HOME**:
  C:\hadoop

- Add the following to the **Path**:
  - %SPARK_HOME%\bin
  - %HADOOP_HOME%\bin
- **SPARK_HOME** (optional):
  C:\spark\spark-3.4.1-bin-hadoop3 (your extracted folder)

5. Verify Installation

Open **Command Prompt** and run:

spark-shell

If successful, you'll see the Spark interactive shell start.

Using scala:

Step 1: Read input text file from local path or Hadoop HDFS.

```
val rdd=spark.sparkContext.textFile("
file:///C:/Users/LENOVO/OneDrive/Desktop/input/input.txt")

val rdd = sc.textFile("hdfs://localhost:9000/krishna/input.txt")
```

rdd collect results:

```
rdd.collect()
```

Step 2: split lines into words.

```
val rdd1 = rdd.flatMap(x => x.split(" "))
```

rdd1 collect results:

```
rdd1.collect()
```

Step 3: Assign count of 1 to each word.

```
val rdd2 = rdd1.map(x => (x, 1))
```

rdd2 collect results:

```
rdd2.collect()
```

Step 4: Aggregate counts by word (reduceByKey)

```
val rdd3 = rdd2.reduceByKey((a, b) => a + b)
```

Step 5:Collect results:

```
rdd3.collect()

rdd3.toDF("word","count").show()
```

Step 6: Save results to Local path or HDFS

```
rdd3.coalesce(1).saveAsTextFile("hdfs://localhost:9000/krishna/input3.txt")

rdd3.coalesce(1).saveAsTextFile("C:\Users\LENOVO\OneDrive\Desktop\Output")
```